



《OpenCV 基础与应用》

实验报告

实验序号： 实验 1

实验名称： HighGUI 编程

姓名： 何宗霖

学号： 23374275

同组： 无

专业： 飞行器控制与信息工程

2025 年 10 月 18 日

一、实验目的：

掌握 OpenCV 的安装与配置，以及掌握 HighGUI 编程。

二、实验内容：

1. 利用 HighGUI 写一个简易的视频播放器，实现视频的播放控制，窗口以自己学号或姓名命名：

- (1) 创建一文档类存放全局相关的数据和变量；
- (2) 鼠标左键控制图像播放或暂停，右键实现图像反转，用滑动条改变播放进度；
- (3) 按“S”或“s”键保存当前图像并另窗显示，按 ESC 或“q”退出程序。

2. 【挑战-选做】拖入文件，如果是图像就显示图像；如果是视频就播放视频；如果直接运行，则从摄像机捕获视频并显示。

三、实验要求：

1. 软件的流程图及说明；
2. 要包含有结果的截屏图像及实验结果详细说明。

四、实验原理：

(1) OpenCV HighGUI 模块：

HighGUI 是 OpenCV 中用于用户界面交互的模块，提供了创建和管理窗口、显示图像/视频、处理鼠标和键盘事件、创建滑动条等功能。

`cv::namedWindow()`用于创建窗口并命名

【完成内容 1：窗口以自己学号姓名命名】

`cv::createTrackbar()`用于创建滑动条。

`cv::setTrackbarPos()`用于设置滑动条位置。

【完成任务（2）：用滑动条改变播放进度】

`cv::imshow()`用于在指定窗口中显示图像。

【完成任务（3）：另窗显示】

`cv::waitKey()`用于等待按键事件，并控制图像刷新闻隔。

`cv::setMouseCallback()`用于注册鼠标事件回调函数。（重点！）

(2) 视频捕获与处理：

`cv::VideoCapture` 类用于从视频文件或摄像头捕获视频帧。

cap.open()用于打开视频文件或摄像头。

【注】：如果打开摄像头后出现报错中断现象，先新建项目测试以下代码，检查设备摄像头是否能够正常开启

```
test.cpp
test (全局范围) main()
#include <iostream>
#include <opencv2/opencv.hpp>

int main() {
    cv::VideoCapture cap;
    cap.open(0, cv::CAP_DSHOW);

    if (!cap.isOpened()) {
        std::cerr << "CRITICAL ERROR: Camera could not be opened!" << std::endl;
        std::cin.get();
        return -1;
    }

    std::cout << "Success! Camera is open. Press 'q' to quit." << std::endl;

    cv::Mat frame;

    // 使用一个无限循环来持续捕获和显示视频帧
    while (true) {
        // 1. 读取新的一帧
        bool success = cap.read(frame);

        // 如果读取失败 (例如摄像头被拔掉), 则退出循环
        if (!success || frame.empty()) {
            std::cerr << "CRITICAL ERROR: Failed to read a frame from the camera!" << std::endl;
            break;
        }

        // 2. 显示这一帧
        cv::imshow("Live Video Feed", frame);

        // 3. 等待一小段时间 (例如30毫秒)
        // 这会给窗口时间来刷新画面, 形成视频效果
        // 同时, 它也会检查用户是否按下了键盘
        // 如果按下的键是 'q' (ASCII码), 就退出循环
        if (cv::waitKey(30) == 'q') {
            break;
        }

        // 循环结束后, 释放资源
        cap.release();
        cv::destroyAllWindows();
    }

    return 0;
}
```

图 1 test 文件代码图

cap.isOpened()用于检查视频源是否成功打开。

cap >> frame 或 cap.read(frame)用于读取下一帧。

cap.get(cv::CAP_PROP_POS_FRAMES)用于获取当前帧的索引。

cap.set(cv::CAP_PROP_POS_FRAMES, pos)用于设置视频播放到指定帧。

cv::bitwise_not()用于实现图像的颜色反转。

【完成任务（2）：右键实现图像反转】

cv::imwrite()用于将图像保存到文件。

【完成额外任务：将另窗展示的图像保存到指定路径】

(3) 事件驱动编程：

程序通过循环不断读取视频帧并显示。

通过 cv::waitKey()捕获键盘事件。

通过 cv::setMouseCallback()注册的函数响应鼠标事件。

通过 cv::createTrackbar()注册的函数响应滑动条事件。

五、实验过程：

(1) OpenCV 环境配置：

在 Visual Studio 2022 中创建 C++空项目。

配置项目【注】：项目路径最好不要有中文

1、release 模式

2、项目-属性-VC++包含目录-include

D:\opencv4.6\opencv\build\include

D:\opencv4.6\opencv\build\include\opencv2

3、项目-属性-库目录-x64-vc15-lib

D:\opencv4.6\opencv\build\x64\vc15\lib

4、C/C++-预处理器

_CRT_SECURE_NO_WARNINGS

5、链接器-输入-附加依赖项-

opencv_world460.lib

6、C/C++-SDL 检查-否

确保 OpenCV 的 DLL 文件（opencv_world460.lib）在系统 PATH 中

(2) 代码实现与调试：

PlayerState 结构体：定义了视频播放器所需的所有状态变量，使其易于在不同函数间共享。

```
struct PlayerState {
    cv::VideoCapture cap;
    cv::Mat currentFrame; //视频当前的帧
    cv::Mat displayFrame; //窗口展示的帧
    bool isPlaying = true; //是否播放
    bool colorInvert = false; //是否翻转
    int sliderPosition = 0; //滑块
    int windowCounter = 1; //窗口计数器
    std::string windowName; //窗口名
};
```

图 2 PlayerState 结构体

onMouse 回调函数：

cv::EVENT_LBUTTONDOWN 检测鼠标左键按下，切换 isPlaying 状态。

cv::EVENT_RBUTTONDOWN 检测鼠标右键按下，切换 colorInvert 状态。

```
void onMouse(int event, int x, int y, int flags, void* userdata) {
    //opencv自己对鼠标的回调函数
    //event: 鼠标左键、右键、移动
    //x,y: 鼠标事件发生时候的像素坐标
    //flags: 鼠标事件发生时是否有键盘修饰键（如 Shift、Ctrl）被按下
    //userdata: 被用来传递一个 PlayerState 对象的指针

    PlayerState* state = static_cast<PlayerState*>(userdata);
    //从一个通用的 void* 指针 (userdata) 恢复出一个具体的 PlayerState* 指针 (state)，即类型转换

    if (event == cv::EVENT_LBUTTONDOWN) {
        state->isPlaying = !state->isPlaying;//反转 “是否播放”
    }
    else if (event == cv::EVENT_RBUTTONDOWN) {
        state->colorInvert = !state->colorInvert;//反转 “是否翻转”
    }
}
```

图 3 onMouse 回调函数

onTrackbar 回调函数：

根据滑动条位置 pos，使用 cap.set(cv::CAP_PROP_POS_FRAMES, pos)设置视频播放到指定帧。

增加了判断，只有当滑动条位置与当前帧位置显著不同时才进行设置。

```
void onTrackbar(int pos, void* userdata) {
    PlayerState* state = static_cast<PlayerState*>(userdata);

    // 只有当滑动条的位置与视频的当前帧位置不匹配时（意味着是用户拖动），才设置视频帧
    if (std::abs(pos - state->cap.get(cv::CAP_PROP_POS_FRAMES)) > 1) {
        state->cap.set(cv::CAP_PROP_POS_FRAMES, pos);
    }
}
```

图 4 onTrackbar 回调函数

main 函数逻辑：

根据 argc (命令行参数数量) 判断是文件模式还是摄像头模式。

文件模式 (argc > 1):

首先尝试使用 cv::imread 打开文件作为图像。如果成功，显示图像并退出。

【选做任务 1: 如果是图像就显示图像】

如果不是图像，则尝试使用 cv::VideoCapture 打开文件作为视频。

【选做任务 2: 如果是视频就播放视频】

同时，视频文件模式下创建滑动条。

摄像头模式(argc == 1):

使用 `state.cap.open(cameraIndex, cv::CAP_DSHOW)` 打开摄像头

(默认 `cameraIndex = 0`)。

摄像头模式下不创建进度条，因为摄像头没有总帧数概念。

主循环：

在 `isPlaying` 为 `true` 时读取帧。

判断 `currentFrame` 是否为空，处理视频结束或无法读取帧的情况。

根据 `colorInvert` 状态对 `currentFrame` 进行颜色反转，结果存储在 `displayFrame`。

`cv::imshow` 显示 `displayFrame`。

处理键盘事件：'q' 或 'ESC' 退出，'s' 保存当前 `displayFrame` 到文件并新窗口显示。

`cv::waitKey(30)` 控制帧率，并捕获按键。

六、实验结果：

1. 软件流程设计图：

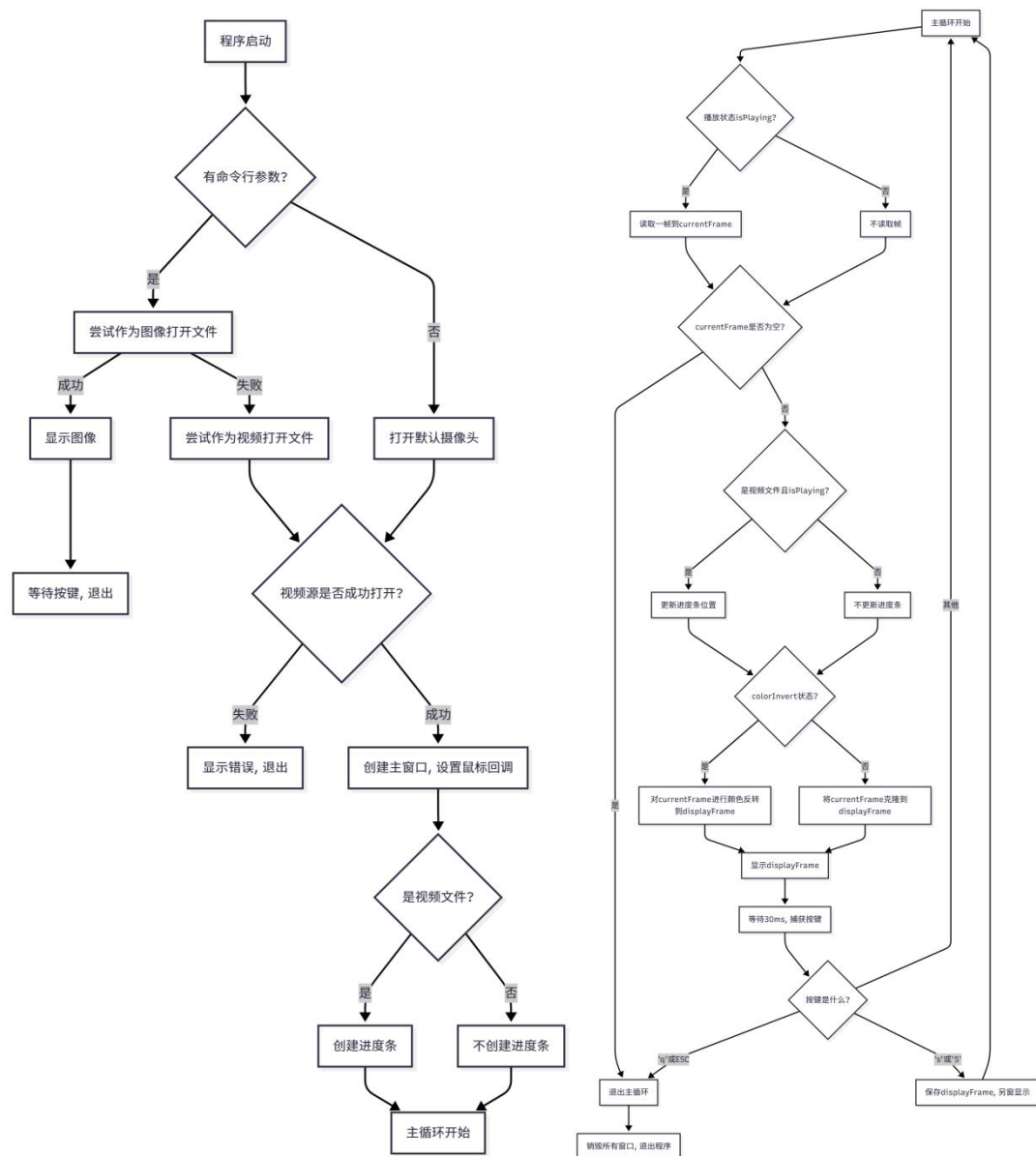


图 5 软件流程设计图

2. 实验结果截图与说明：

(1) 运行视频文件"D:\芯片使用视频\去胶.mp4"或"D:\芯片使用视频\曲线图.mp4"

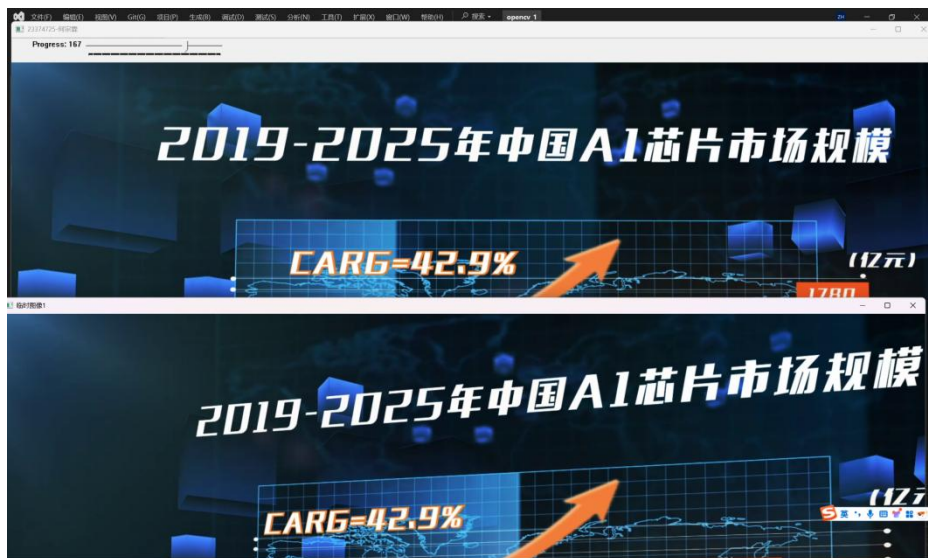


图 6 视频模式效果展示图 1

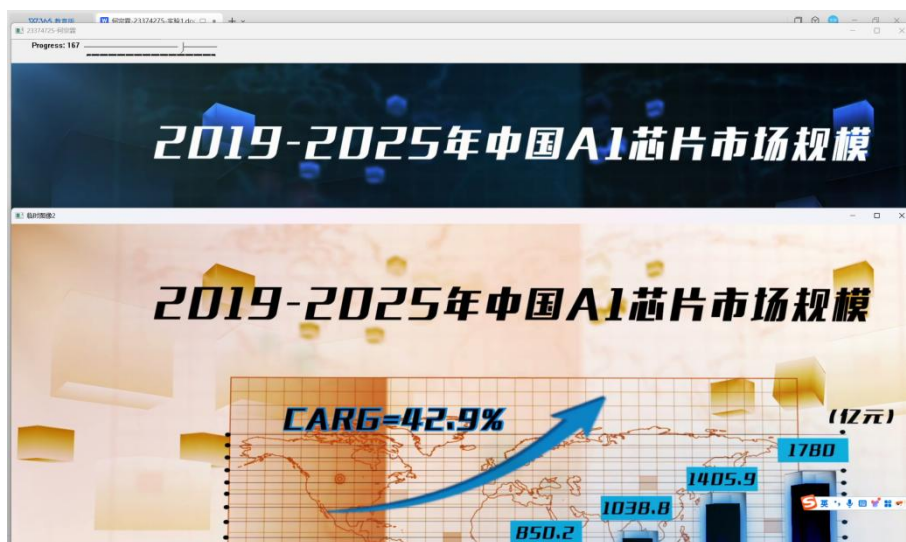


图 7 视频模式效果展示图 2



图 8 视频模式效果展示图 3

效果图 1:

展示了窗口正确命名为: 23374275-何宗霖

展示了保存键盘's'或'S'保存当前帧并另窗显示功能

效果图 2:

展示了鼠标右键点击实现图像颜色反转功能

效果图 3:

展示了用滑动条改变播放进度功能

(2) 运行图像文件"D:\opencv_zixue\lena.jpg"



图 9 图像模式效果展示图

(3) 直接运行 (摄像头模式)

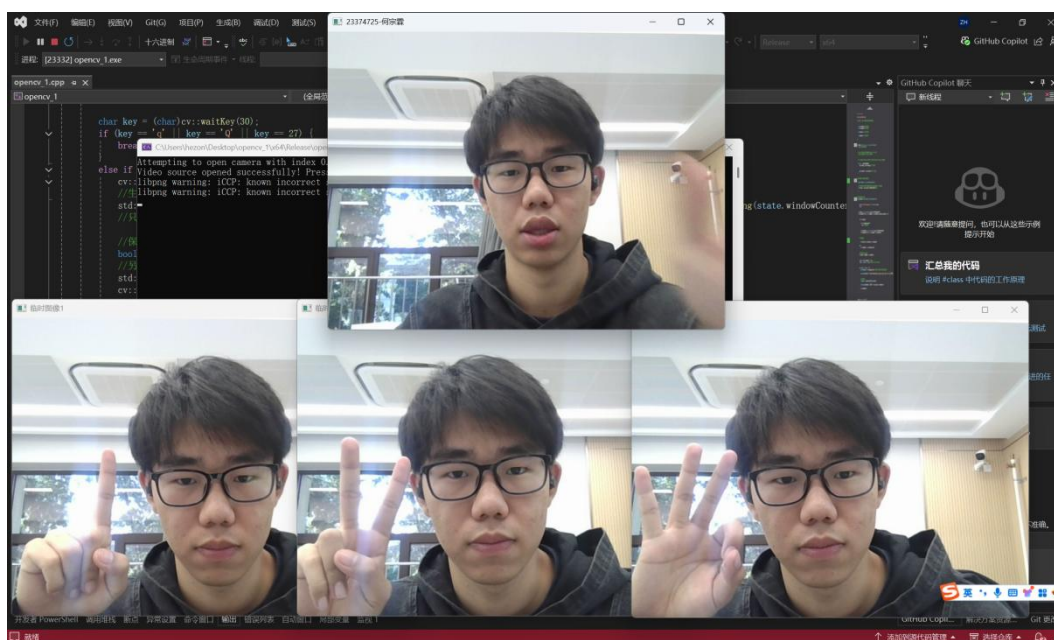


图 10 摄像头模式效果展示图 1

第一行的图片：

展示了窗口正确命名为：23374275-何宗霖

展示了鼠标左键的暂停功能，图像右下角为手部操作瞬间影像

第二行三张图片：

展示了键盘's'或'S'保存当前帧并另窗显示功能，分别命名为：临时图像 1-3

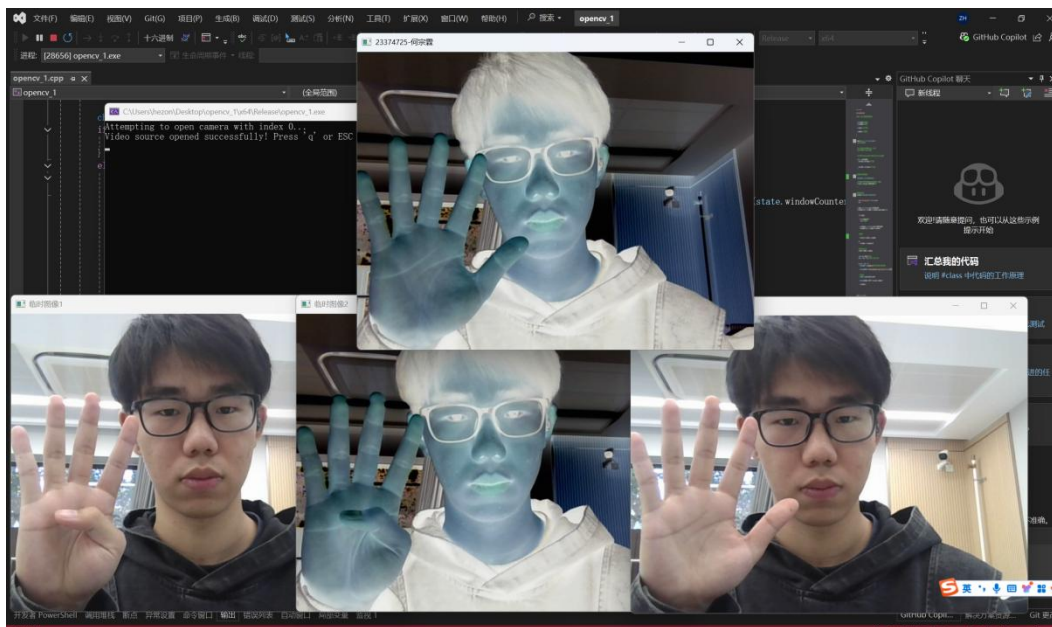


图 11 摄像头模式效果展示图 2

这组图片展示了鼠标右键点击实现图像颜色反转的功能

七、结果分析与实验结论：

1.结果分析：

本次实验成功实现了基于 OpenCV HighGUI 模块的简易视频播放器，并完成了挑战性的文件拖拽和摄像头支持功能。

主要实现功能：

(1) 全局状态管理: PlayerState 结构体有效封装了所有与播放器状态相关的变量，使得代码结构清晰，便于管理和传递数据。

(2) 用户交互：

鼠标控制: 鼠标左键点击实现播放/暂停的切换, 右键点击实现图像颜色反转, 响应灵敏, 符合直观操作。

滑动条控制: 进度条功能在视频文件模式下表现良好, 用户可以精确地控制播放进度。通过判断滑动条位置与当前帧的差异来避免不必要的

`set(CAP_PROP_POS_FRAMES)` 调用，优化了性能。

键盘控制：'q'或'ESC'退出程序，'s'或'S'保存当前帧并另窗显示，满足实验要求。保存图像在指定文件夹下，方便查看。

(3) 媒体文件处理：

程序能够根据命令行参数智能判断输入是图像文件还是视频文件。

如果是图像，则直接显示图像并退出。

如果是视频，则进入视频播放模式，具备完整的播放器功能。

摄像头支持：当程序无命令行参数运行时，能自动打开默认摄像头，并提供播放/暂停、反转、保存等功能，拓展了播放器的应用场景。

(4) 遇到的问题与解决方案：

滑动条与视频播放不同步：初始时可能存在滑动条位置与视频实际播放帧不同步的问题。通过在 `onTrackbar` 中增加 `std::abs(pos - state->cap.get(cv::CAP_PROP_POS_FRAMES)) > 1` 的判断，确保只在用户主动拖动滑动条且位置发生显著变化时才设置视频帧，有效解决了此问题。

摄像头打开失败：先利用 `test` 代码测试设备摄像头是否正常，再在代码中，`state.cap.open(cameraIndex, cv::CAP_DSHOW)` 使用 `CAP_DSHOW` 标志来强制使用 `DirectShow` 接口，这在 Windows 上通常更为稳定和兼容。最后确保自己项目的路径中没有中文（本人就是因此导致运行中断，无法开启摄像头）

保存图片的文件名和路径：为了避免文件覆盖，实现多次保存关键帧，在 `structure` 中增添了属性 `windowCounter`，并初始化为 1。采用了 `std::to_string(state.windowCounter)` 动态生成文件名。同时，为了规范保存路径，指定了绝对路径 `"C:\\Users\\hezon\\Desktop\\opencv_1\\opencv_1\\save_img\\"`。

2. 实验结论。

本次实验不仅巩固了 OpenCV 的安装与配置知识，更深入地理解了 HighGUI 模块在用户界面交互中的重要作用。通过实践开发，掌握了事件驱动编程的思想，以及如何利用 OpenCV 提供的 API 实现复杂的媒体处理功能。特别是挑战部分的完成，展现了程序处理不同输入源的灵活性和鲁棒性。实验结果表明，所编写的程序功能完善，用户体验良好，达到了实验的预期目标